Universidad Don Bosco Dirección de Educación a Distancia



Electrónica Aplicada al Internet de las Cosas EIC901

Docente: Eduardo Antonio Henríquez Martínez

Estudiantes:

Christian Geovanni Centeno, CS241743

José Alexander Montoya, MQ252529

Gabriel Quintanilla Rodríguez, QR230082

Actividad:

Etapa 3: Foro 1

Introducción

El proyecto Panadería Rosario corresponde al desarrollo de una aplicación móvil orientada a apoyar la gestión de clientes, catálogo, pedidos, inventario, delivery y reportes de una panadería. En esta etapa, el núcleo funcional implementado previamente en Kotlin se integra en una aplicación Android con una interfaz construida mediante Jetpack Compose. La aplicación utiliza una sola actividad principal, navegación entre pantallas, componentes Material 3 y estado observable para mantener la interfaz sincronizada con las operaciones del sistema.

Jetpack Compose es el toolkit moderno de Android para construir interfaces nativas mediante un enfoque declarativo. En lugar de modificar vistas de forma imperativa, la interfaz se describe a partir del estado actual de la aplicación y Compose vuelve a ejecutar las funciones componibles cuando cambian los valores observados. Este modelo permite reducir la manipulación manual de vistas y organizar la UI alrededor de funciones y estados.

El objetivo de este informe es describir los fundamentos de Jetpack Compose y analizar su aplicación concreta en Panadería Rosario. Para ello se estudian el modelo declarativo, las funciones @Composable, el manejo de estado, Material 3 y Navigation Compose, y posteriormente se relacionan estos conceptos con los componentes implementados en el proyecto.

Objetivos

Objetivo general

Analizar el uso de Jetpack Compose en el desarrollo de la aplicación móvil Panadería Rosario y documentar su integración con la lógica de negocio implementada en Kotlin.

Objetivos específicos

- Describir el modelo declarativo de Jetpack Compose y el funcionamiento de las funciones componibles.
- Explicar el manejo de estado y la recomposición utilizados en las pantallas de la aplicación.
- Identificar los componentes Material 3 empleados para formularios, listas, tarjetas y acciones.
- Explicar la navegación entre cliente, administración, carrito, delivery, pedidos y reportes.
- Relacionar la capa de interfaz con los modelos y servicios orientados a objetos del proyecto.

Jetpack Compose como framework de interfaz

Modelo declarativo y funciones componibles

Compose utiliza un paradigma declarativo: una función describe cómo debe verse la interfaz para un estado determinado. Cuando el estado cambia, Compose puede recomponer las funciones que dependen de ese valor y actualizar únicamente lo necesario. Android Developers (s. f.-a) contrasta este enfoque con la modificación imperativa de vistas, en la cual el programador debe localizar componentes y alterar sus propiedades manualmente.

La unidad básica de la interfaz es la función anotada con @Composable. En Panadería Rosario, cada pantalla principal se implementa como una función componible independiente. Entre ellas se encuentran LoginScreen, RegistroScreen, CatalogoScreen, CarritoScreen, EntregaScreen, MisPedidosScreen, AdminProductosScreen, AdminPedidosScreen, DeliveryScreen y ReporteScreen. Esta distribución permite que cada pantalla mantenga una responsabilidad visual concreta.

```
@Composable
fun LoginScreen(...)

@Composable
fun CatalogoScreen(...)

@Composable
fun EntregaScreen(...)

@Composable
fun AdminProductosScreen(...)
```

Punto de entrada de la aplicación

La aplicación utiliza MainActivity como actividad principal. En su método onCreate se crea una instancia de AppState y setContent entrega el control visual a PanaderiaApp. Esta estructura mantiene a MainActivity ligera y evita concentrar la construcción de pantallas dentro de la actividad.

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        val appState = AppState()
        setContent {
            PanaderiaApp(appState)
        }
    }
}
```

Manejo de estado y recomposición

El estado representa información que puede cambiar durante la ejecución. La documentación oficial indica que mutableStateOf crea un estado observable integrado con el runtime de Compose; cuando cambia un valor leído por una función componible, se programa su recomposición. En Panadería Rosario, AppState actúa como contenedor de estado compartido y como punto de coordinación entre la interfaz y los servicios del negocio.

```
var clienteActual by mutableStateOf<Cliente?>(null)
    private set

var esAdministrador by mutableStateOf(false)
    private set

var versionDatos by mutableStateOf(0)
    private set
```

La variable versionDatos se incrementa después de operaciones que modifican datos relevantes, por ejemplo al registrar un cliente, modificar cantidades del carrito, confirmar un pedido, avanzar un estado o actualizar productos. Las pantallas que leen este valor se recomponen y vuelven a consultar los servicios, lo que permite reflejar los cambios en la interfaz.

En los formularios se utiliza remember junto con mutableStateOf para mantener temporalmente entradas como nombre, teléfono, dirección, usuario, contraseña, precios y datos de delivery. Este uso coincide con el propósito de remember: conservar valores mientras el elemento permanezca en la composición.

```
var receptor by remember { mutableStateOf(cliente?.nombre.orEmpty()) }
var telefono by remember { mutableStateOf(cliente?.telefono.orEmpty()) }
var direccion by remember { mutableStateOf(cliente?.direccion.orEmpty()) }
var referencias by remember { mutableStateOf("") }
```

Como consideración técnica, la versión actual mantiene los datos principales en memoria. Para una evolución posterior sería conveniente utilizar un ViewModel y una fuente de persistencia, ya que la documentación de Android recomienda elevar el estado que contiene lógica de negocio a contenedores observables adecuados y conservar una única fuente de verdad.

Diseño con Material 3 y componentes Compose

Panadería Rosario incorpora la dependencia androidx.compose.material3:material3 y define un tema propio mediante MaterialTheme. Material 3 organiza el sistema visual alrededor de esquemas de color, tipografía y formas, y sus componentes adoptan automáticamente los valores definidos por el tema.

```
private val PanaderiaColors = lightColorScheme(
    primary = Color(0xFF6D4025),
    secondary = Color(0xFF9B6A45),
    background = Color(0xFFFFFBF6),
    surface = Color(0xFFF8EFE5)
)
```

La combinación de marrones y tonos crema busca mantener una identidad visual relacionada con un negocio de panadería. En las pantallas se utilizan componentes como Column, Row, Button, OutlinedButton, OutlinedTextField, Card, IconButton, FilterChip, LazyColumn, Text y MaterialTheme.

LazyColumn se utiliza en listados de productos y pedidos. Este componente crea y posiciona principalmente los elementos visibles en el área de desplazamiento, lo que resulta adecuado para listas que pueden crecer durante el uso de la aplicación. Los formularios de autenticación, registro, edición y delivery utilizan OutlinedTextField para capturar datos del usuario.

Navegación con Navigation Compose

La navegación se concentra en PanaderiaApp mediante rememberNavController y NavHost. En Navigation Compose, el NavHost muestra destinos componibles asociados a rutas y el NavController administra la navegación y la pila de destinos. Esta decisión mantiene la lógica de navegación fuera de las pantallas individuales: las pantallas reciben funciones callback como onCarrito, onPedidos, onVolver u onDelivery.

```
val nav = rememberNavController()

NavHost(navController = nav, startDestination = "login") {
    composable("login") { ... }
    composable("catalogo") { ... }
    composable("carrito") { ... }
    composable("datos_delivery") { ... }
    composable("admin") { ... }
    composable("admin_productos") { ... }
    composable("admin_pedidos") { ... }
    composable("admin_delivery") { ... }
    composable("reporte") { ... }
}
```

La arquitectura usa una sola actividad y varias funciones componibles como destinos. Android Developers (s. f.-g) señala que NavHost es el host correspondiente a Compose y que NavController funciona como coordinador central de la navegación. En Panadería Rosario también se usa popBackStack para regresar a la pantalla anterior y popUpTo para limpiar destinos después de iniciar o cerrar sesión.

Aplicación de Compose en Panadería Rosario

Autenticación y registro

La pantalla inicial diferencia el acceso del cliente y el acceso administrativo. El cliente ingresa con su número de teléfono y, si todavía no está registrado, puede abrir RegistroScreen. El registro solicita nombre, teléfono y dirección y utiliza Validador para impedir campos vacíos. El administrador dispone de una pantalla de acceso independiente.

Catálogo, inventario y carrito

CatalogoScreen presenta los productos de la panadería, permite filtrar por categoría y agregar unidades al carrito. AppState controla las cantidades agregadas y modifica el stock disponible. Al aumentar o disminuir cantidades, el inventario y el carrito se actualizan durante la misma ejecución. Si el usuario cierra sesión con productos sin confirmar, las unidades reservadas se devuelven al stock.

CarritoScreen muestra producto, cantidad, subtotal y total. Desde esta pantalla el usuario puede modificar cantidades y continuar al formulario de delivery antes de confirmar definitivamente el pedido.

Delivery y seguimiento de pedidos

El flujo de compra incorpora EntregaScreen, donde el cliente confirma la persona que recibe, teléfono de contacto, dirección y referencias. Esta información se encapsula en la clase Entrega y queda asociada al objeto Pedido. De esta manera la dirección de entrega puede diferir de los datos de registro del cliente.

```
data class Entrega(
    val receptor: String,
    val telefonoContacto: String,
    val direccion: String,
    val referencias: String = ""
)
```

PedidoService controla el ciclo de vida del pedido y restringe los cambios de estado a la secuencia RECIBIDO -> EN_PREPARACION -> EN_CAMINO -> ENTREGADO. El módulo administrativo DeliveryScreen filtra los pedidos EN_CAMINO y muestra la información necesaria para completar la entrega. El administrador puede marcar el pedido como ENTREGADO desde este módulo.

Gestión administrativa de productos

AdminProductosScreen permite crear y editar productos, seleccionar categoría, cambiar precio, actualizar stock y eliminar registros. La edición reutiliza el formulario principal: al seleccionar un producto, sus valores se cargan en los campos y luego se guardan mediante las funciones de ProductoService. De este modo el CRUD desarrollado en la lógica de negocio dispone ahora de una representación gráfica en Compose.

Reportes

ReporteScreen combina un resumen textual con un gráfico de barras construido con composables. El gráfico cuenta los pedidos en cada EstadoPedido y representa la proporción mediante Box y fillMaxWidth. ReporteService calcula productos registrados, disponibilidad, unidades vendidas, pedidos por estado, ventas acumuladas entregadas y producto más vendido. Además, permite exportar el reporte como PDF a la carpeta Descargas mediante PdfDocument y MediaStore.

Integración con la Programación Orientada a Objetos

Jetpack Compose funciona como capa de presentación y no reemplaza la lógica orientada a objetos del proyecto. La aplicación conserva una separación entre modelos, servicios, utilidades y pantallas. Esta organización permite que las funciones componibles soliciten operaciones a AppState y a los servicios en lugar de implementar directamente las reglas del negocio.

Elemento	Aplicación en el proyecto
Clases y objetos	Producto, Pedido, DetallePedido, Entrega, Cliente y Administrador representan entidades del negocio.
Herencia	Cliente y Administrador derivan de Usuario.
Interfaces	Pedido implementa Calculable; MantenimientoBase<T, ID> define operaciones reutilizables para servicios.
Encapsulamiento	Los servicios concentran operaciones de productos, clientes, pedidos y reportes.
Colecciones	Los servicios usan colecciones mutables para mantener productos, clientes y pedidos durante la ejecución.

Esta separación también favorece la mantenibilidad: la UI puede cambiar sin reescribir las reglas de PedidoService, ProductoService o ReporteService, y las reglas del negocio pueden evolucionar sin concentrarse en MainActivity.

Correspondencia con los requisitos de la Etapa 3

Requisito	Implementación observada
Pantalla de autenticación funcional	LoginScreen, AdminLoginScreen y RegistroScreen permiten acceso y registro con validaciones.
Módulo principal con Jetpack Compose	Las pantallas de cliente y administración están implementadas como funciones @Composable.
Navegación entre pantallas	PanaderiaApp usa rememberNavController, NavHost y rutas para catálogo, carrito, delivery, pedidos, administración y reportes.
Validación de formularios	Validador y comprobaciones con require controlan campos vacíos, datos de pedido y delivery.
Aplicación de lógica del negocio	AppState enlaza Compose con ProductoService, ClienteService, PedidoService y ReporteService.
Gestión y seguimiento de pedidos	Estados controlados desde RECIBIDO hasta ENTREGADO y módulo específico para pedidos EN_CAMINO.
Reportes	Resumen, gráfico por estado y exportación a PDF.

Validaciones y manejo de errores

La aplicación aplica validaciones en dos niveles. En la interfaz, los formularios comprueban que los campos requeridos tengan contenido y muestran mensajes mediante ErrorText. En la lógica de negocio, PedidoService utiliza require para evitar pedidos vacíos, cantidades no válidas y productos no disponibles. También se impide saltar arbitrariamente entre estados del pedido. Estas validaciones permiten que las reglas principales se mantengan fuera de la presentación.
En el flujo de delivery se exige receptor, teléfono de contacto y dirección antes de confirmar el pedido. Los errores se capturan en la interfaz y se muestran al usuario sin cerrar la aplicación.

Limitaciones actuales y mejoras futuras

La versión actual mantiene productos, clientes y pedidos principalmente en memoria. Esto permite demostrar la lógica y la interacción entre pantallas, pero la información puede perderse al terminar el proceso de la aplicación.

El acceso administrativo utiliza credenciales definidas dentro de la aplicación y debe considerarse un mecanismo de prueba para la etapa académica, no una solución final de autenticación.

Como evolución del proyecto se recomienda incorporar persistencia local o remota, un ViewModel para separar mejor el estado de la actividad, autenticación segura, pruebas de UI y navegación, y una estrategia de almacenamiento de datos acorde con el uso real de la aplicación.

Conclusiones

Jetpack Compose permitió transformar la lógica de Panadería Rosario en una aplicación Android con una interfaz declarativa y modular. Las pantallas fueron separadas por responsabilidad y la navegación se centralizó en PanaderiaApp, mientras que AppState funciona como enlace entre la UI y los servicios del negocio.

El uso de mutableStateOf y remember permite que los formularios y datos visibles reaccionen a cambios durante la ejecución. Material 3 aporta componentes reutilizables para construir formularios, listas, tarjetas, botones y una identidad visual consistente. Navigation Compose, por su parte, organiza el flujo entre autenticación, catálogo, carrito, delivery, pedidos, administración y reportes.

La aplicación mantiene la lógica orientada a objetos previamente desarrollada y la amplía con componentes propios del entorno Android. El resultado cubre los flujos principales del negocio: clientes, catálogo, inventario, pedidos, delivery, administración y reportes. Aunque todavía existen oportunidades de mejora en persistencia y arquitectura de estado, la etapa actual establece una base funcional para continuar el desarrollo de Panadería Rosario.

Referencias

- Android Developers. (s. f.-a). Thinking in Compose. https://developer.android.com/develop/ui/compose/mental-model
- Android Developers. (s. f.-b). State and Jetpack Compose. https://developer.android.com/develop/ui/compose/state
- Android Developers. (s. f.-c). Compose UI architecture. https://developer.android.com/develop/ui/compose/architecture
- Android Developers. (s. f.-d). Material Design 3 in Compose. https://developer.android.com/develop/ui/compose/designsystems/material3
- Android Developers. (s. f.-e). Lazy lists and lazy grids. https://developer.android.com/develop/ui/compose/lists
- Android Developers. (s. f.-f). Navigate between screens with Compose. https://developer.android.com/codelabs/basic-android-kotlin-compose-navigation
- Android Developers. (s. f.-g). Navigation. https://developer.android.com/guide/navigation